

Stabilizing MoE Reinforcement Learning by Aligning Training and Inference Routers

Wenhan Ma¹ Hailin Zhang² Liang Zhao² Yifan Song^{1,2} Yudong Wang^{1,2} Fuli Luo² Zhifang Sui¹

Abstract

Reinforcement learning (RL) has emerged as a crucial approach for enhancing the capabilities of large language models. However, in Mixture-of-Experts (MoE) models, RL training tends to be unstable and can even collapse entirely. We analyze the training-inference consistency of MoE models and identify a notable discrepancy in routing behaviors between the two phases. To address this issue, we propose **Rollout Routing Replay (R3)**, a novel and effective method that records routing distributions from the inference engine and replays them during training. R3 significantly reduces training-inference policy KL divergence and mitigates extreme discrepancies without compromising training speed. Extensive experiments on various settings confirm that R3 succeeds in stabilizing RL training, preventing collapse and outperforming strong baselines. R3 is orthogonal to most policy optimization algorithm improvements, allowing it to be used in conjunction with them. We believe this work can offer a new solution for stabilizing RL in MoE model.

1. Introduction

Reinforcement learning (RL), especially reinforcement learning with verifiable rewards (RLVR), has become a cornerstone in the post-training of large language models (LLMs) (Ouyang et al., 2022; OpenAI, 2024; Guo et al., 2025). By leveraging large-scale RL, LLMs acquire the advanced capabilities necessary to tackle complex problems, including competition-level mathematics (Guo et al., 2025) and practical code agent tasks (Luo et al., 2025a), through more profound and extended reasoning.

¹State Key Laboratory of Multimedia Information Processing, School of Computer Science, Peking University ²LLM Core, Xiaomi. Correspondence to: Zhifang Sui <szf@pku.edu.cn>, Fuli Luo <luofuli@pku.edu.cn>.

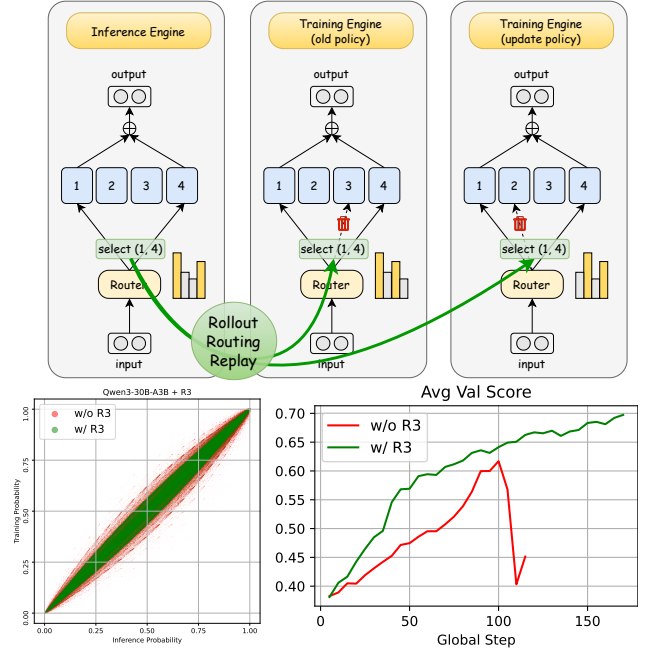


Figure 1. Top: Illustration of the Rollout Routing Replay (R3). Bottom left: Training and inference discrepancies before and after applying R3. Bottom right: Reinforcement learning training performance before and after applying R3.

A key challenge is enhancing the stability of the LLM RL training process. Modern RL frameworks typically employ distinct engines for inference and training phases (e.g., SGLang (Zheng et al., 2024) for inference and Megatron (Shoeybi et al., 2019) for training). This architectural separation can lead to divergent token probabilities, potentially causing catastrophic collapse (He & Lab, 2025). To mitigate this discrepancy, Yao et al. (2025) incorporate importance sampling mechanism in policy updating, while He & Lab (2025) introduce specialized compute kernels to reduce nondeterminism during LLM Inference. However, in practice, existing approaches do not fully resolve the intensified off-policy issue that arises during RL training on Mixture-of-Experts (MoE) models.

In this work, we identify the router as a key driver of instability in MoE-based RL. In MoE models, a router dynamically selects and activates a subset of experts for each token. Be-

cause routing decisions can vary across inference/training framework, MoE models often exhibit substantially larger policy discrepancies between rollout and optimization than dense models. Rather than relying on heuristics such as filtering out tokens with large discrepancies (Zhao et al., 2025), we address the problem at its source by explicitly controlling the routing behavior.

To this end, we propose **Rollout Routing Replay (R3)**, a novel and effective method for stabilizing RL training of MoE models. R3 captures routing distributions from the inference engine during sequence generation and replays them into the training engine. This process reduces the discrepancy between training and inference, leading to a significant reduction in the KL divergence between training and inference policies. By replaying the routing distributions, we can precisely optimize only the experts involved in generating the token, avoiding unnecessary optimization of unused experts that would introduce gradient noise.

We conduct experiments across multiple MoE model families (Qwen3 (Yang et al., 2025), DeepSeek-V2-Lite (DeepSeek-AI et al., 2024)), model variants (Base, Instruct-SFT, Reasoning-SFT) and RL tasks (Math, Software engineering). Results show that R3 improves training stability, prevents collapse, and boosts evaluation performance, surpassing strong baseline methods like GSPO and TIS. Speed and memory analysis reveals only a 3.45% slowdown and manageable memory overhead, demonstrating strong practicality for large-scale training.

We summarize the contributions of our work as follows: (a). We systematically identify and analyze routing distribution discrepancies between training and inference in MoE models, highlighting their role in training instability. (b). We proposed Rollout Routing Replay, which reuses inference routing distributions inside the training engine to align routing behavior between training and inference. (c). We apply R3 in multiple settings for MoE reinforcement learning and show that R3 outperforms baseline methods in terms of stability and overall performance.

2. Preliminaries

2.1. Nondeterminism in Modern LLM framework

Modern LLM training and inference framework often exhibit nondeterministic behavior; this is observed not only across different frameworks but also within the same framework. Importantly, such variability is not primarily attributable to mismatched mathematical formulations (e.g., dropout or batch normalization, which are typically not present in modern LLMs). Instead, it arises from numerical effects inherent to finite-precision computation and low-level implementation details. The main contributing factors are as follows:

Floating-point addition is non-associative: Floating-point addition is non-associative, meaning that the result depends on the order of summation. Limited precision and rounding errors can cause discrepancies when the execution order changes, a problem that becomes especially pronounced in iterative computations and parallel reductions.

Variation in operator and kernel selection: High-level frameworks (e.g., Torch) may dynamically select different low-level kernels (e.g., CUDA) at runtime based on hardware, batch size, or data type. Although these kernels are mathematically equivalent, they often execute floating-point operations in different orders, which can lead to slight variations in the results.

Violation of batch invariance: As stated in (He & Lab, 2025), in asynchronous inference systems, samples may enter and leave dynamically, causing the batch size to vary over time. This can trigger different backend kernel selections, leading to nondeterministic inference results.

MoE Router Amplifies Nondeterminism: MoE models use a router with a Top-K operation, where tiny numerical differences can change the selected Top-K experts and thus alter the model’s computation path. This discrete switch can amplify floating-point perturbations into significant output changes, undermining the stability of reinforcement learning. The proposed R3 method is designed to mitigate this issue.

2.2. Policy Optimization Algorithm in Reinforcement Learning of LLM

Recently, reinforcement learning has been employed to train large language models in order to enhance their reasoning capabilities. Popular algorithms in this area include GRPO (Shao et al., 2024), DAPO (Yu et al., 2025) and GSPO (Zheng et al., 2025), all of which are derived from the classical Proximal Policy Optimization (PPO) (Schulman et al., 2017) algorithm. However, a critical issue arises from the common practice of using different frameworks for sampling and training: data is sampled via the inference framework, but the loss is computed using the training framework. We denote the output probability distributions of the inference and training frameworks as μ and π . Consequently, the actual optimization objective becomes:

$$\mathcal{J}_{\text{PPO}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \mu_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{|y|} \sum_{t=1}^{|y|} \min \left(w_t \hat{A}_t, \text{clip}(w_t, 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right]. \quad (1)$$

The importance sampling ratio w_t for the token y_t in the sequence y is defined as:

$$w_t = \pi_{\theta}(y_t|x, y_{<t}) / \pi_{\theta_{\text{old}}}(y_t|x, y_{<t}) \quad (2)$$

This policy discrepancy may drive training instability in reinforcement learning. In MoE models, this problem becomes further exacerbated, as it can result in the loss being computed with experts that were not utilized during sampling, thereby causing more severe policy divergence. Moreover, during parameter optimization, gradients may erroneously propagate to experts that did not participate in sampling, leading to unstable optimization.

3. Training-Inference Discrepancies in MoE Models

Training-inference discrepancies in RL frameworks frequently lead to unstable training. In this section, we demonstrate that this policy discrepancy is significantly amplified in MoE models, primarily stemming from inconsistent routing distributions.

3.1. Policy Discrepancies Analysis

Here we employ an MoE model, Qwen3-30B-A3B (Yang et al., 2025), for our experiments to analyze the policy discrepancy between the training and inference engines. First, we use the SGLang inference engine to generate responses for 2,048 mathematics problems, saving the token probabilities for each generated token. This process yields about 20 million response tokens. Then these responses are passed through Megatron to obtain the corresponding probabilities assigned by the training engine. We use several metrics to quantify the divergence between these two probability distributions.

KL Divergence Estimation Let T be the set of all response tokens, We use the $k3$ method (Schulman, 2020) to estimate the KL divergence between the training and inference distributions at token-level. The estimation formula is given as follows:

$$\mathcal{D}_{\text{KL}}(\mu_\theta; \pi_\theta) \approx \frac{1}{|T|} \sum_{t \in T} \left[\frac{\pi_\theta(t)}{\mu_\theta(t)} - 1 - \log \frac{\pi_\theta(t)}{\mu_\theta(t)} \right]. \quad (3)$$

Our calculations show that the estimated KL divergence of Qwen3-30B-A3B (MoE) is 1.535×10^{-3} , while that of Qwen3-8B (Dense baseline) is 6.4×10^{-4} .

Visualization To visualize the training-inference discrepancies of the MoE model, we randomly sample 10 million response tokens and plot a scatter diagram with SGLang probabilities on the x-axis and Megatron probabilities on the y-axis. The degree of concentration around the $y = x$ line indicates the degree of consistency. As shown in Figure 2a and 2c, compared to Qwen3-8B, Qwen3-30B-A3B exhibits a much wider scatter band, revealing a larger training-inference discrepancy.

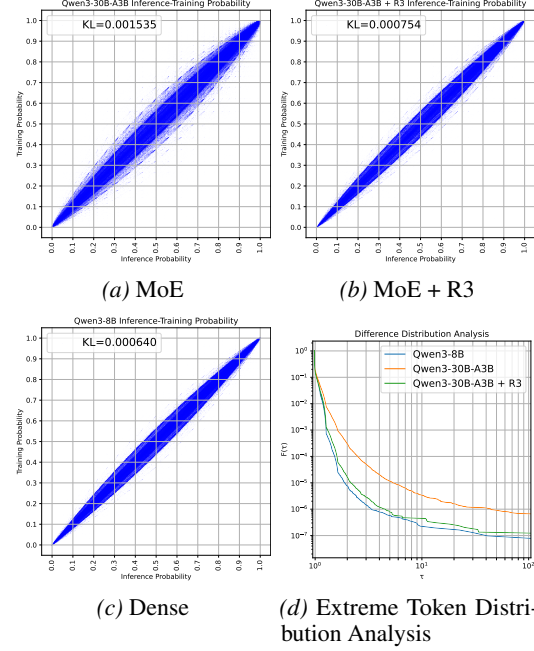


Figure 2. (a): Illustration of the training-inference discrepancy in the MoE model. (b): Illustration of the training-inference discrepancy in the MoE+R3 model. (c): Illustration of the training-inference discrepancy in the Dense model. (d): Extreme Token Distribution Function, calculated based on Equation 4.

Extreme Token Distribution Analysis To quantify the discrepancy between the model’s behavior during training and inference, we introduce the Extreme Token Distribution Function, $F(\tau)$, defined as:

$$F(\tau) = \frac{1}{|T|} \sum_{t \in T} \mathbf{I} \left[\max \left(\frac{\pi_\theta(t)}{\mu_\theta(t)}, \frac{\mu_\theta(t)}{\pi_\theta(t)} \right) > \tau \right]. \quad (4)$$

This function measures the proportion of *extreme tokens*—those for which the probability ratio between the training distribution π_θ and the inference distribution μ_θ surpasses a threshold τ . Figure 2d plots this function $F(\tau)$ against the threshold τ . The plot reveals that for $\tau > 2$, the Qwen3-30B-A3B model exhibits a fraction of extreme tokens an order of magnitude larger than that of the Qwen3-8B model. This significant gap indicates a substantially higher degree of training-inference discrepancy in the MoE model.

3.2. Routing Discrepancies Analysis

From the perspective of functional continuity, the key difference between MoE and Dense models lies in the non-continuity introduced by routing. In MoE models, small perturbations in the router input can lead to entirely different experts being selected, causing large changes in layer outputs. Dense models, lacking an explicit expert selection process, do not exhibit this phenomenon.

Based on this, we further analyze router distribution discrep-

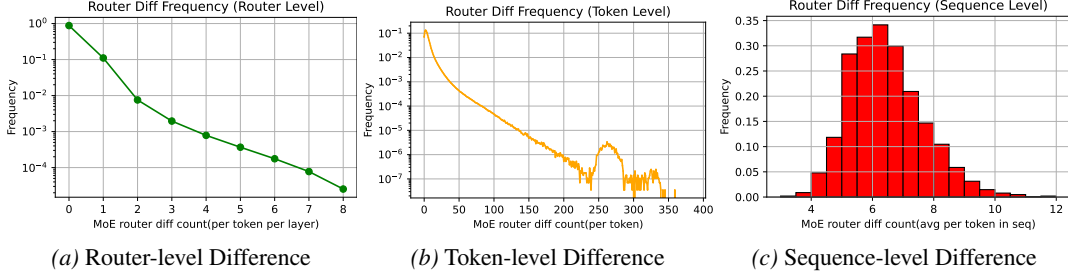


Figure 3. Router discrepancy analysis

anecies between training and inference in MoE models. We use SGLang (Zheng et al., 2024) to generate responses for 2048 mathematical problems with Qwen3-30B-A3B (Yang et al., 2025). For each response, we collect the routing distribution of all tokens (including input tokens) from the inference engine. Then, we feed these sequences into the Megatron engine for forward propagation. This process yields the routing distribution observed from training engine. We compare these two sets of routing information at different levels:

Router-level Comparison: For each token and each MoE layer, we count the number of differing expert choices made by the MoE Router and calculate the frequency of these differences. Figure 3a illustrates the results. It is observed that approximately 10% of the routers select different experts during training compared to inference.

Token-level Comparison: For each token, we count the number of differing expert choices made by the MoE Router across all layers. We also determine the frequency of these occurrences. Figure 3b presents these findings. It shows that 94% of tokens select a different expert in at least one layer during the forward pass.

Sequence-level Comparison: For a sequence, we compute the router distribution difference of each token, then average over tokens to obtain the mean difference per sequence and plot a histogram 3c. Results show that the mean difference per token is approximately 6 routers.

These findings demonstrate that during training and inference, MoE models exhibit router distribution discrepancies. In 4 we show empirically that routing discrepancies is the main contributor to the additional training-inference discrepancies of MoE compared to Dense models.

4. Rollout Routing Replay

This section provides a detailed description of the implementation of Rollout Routing Replay (R3), its caching support in multi-turn dialogue, and an analysis of its effects on training-inference discrepancies.

4.1. Implementation

We first describe the conventional forward pass of a MoE layer within a training framework. Consider a sequence s at the t -th token and the MoE layer in the l -th Transformer block. Let the input to this layer during training be $\mathbf{x}_{\text{train}}$. The router logits are calculated by:

$$\mathbf{s}_{\text{train}} = \mathbf{x}_{\text{train}} \mathbf{W}_r, \quad (5)$$

where $\mathbf{W}_r$ denotes the router’s linear weight matrix. Let the number of experts be M and the number of experts to select be K . During training, the router selects the top- K experts based on the logits. This is represented by a binary mask:

$$\mathbf{I}_{\text{train}} = \text{TopKMask}(\mathbf{s}_{\text{train}}, K), \quad (6)$$

where $\mathbf{I}_{\text{train}} \in \{0, 1\}^M$ and $\sum_i I_{\text{train},i} = K$. The gating weights are then produced by applying a softmax over the logits of the selected experts:

$$g_{\text{train},i} = \frac{I_{\text{train},i} \exp(s_{\text{train},i})}{\sum_{j=1}^M I_{\text{train},j} \exp(s_{\text{train},j})} \quad \text{for } i = 1, \dots, M. \quad (7)$$

Finally, the MoE layer’s output is computed as a weighted sum of the expert outputs:

$$\mathbf{y}_{\text{train}} = \sum_{i=1}^M g_{\text{train},i} \mathcal{E}_i(\mathbf{x}_{\text{train}}), \quad (8)$$

where $\mathcal{E}_i(\cdot)$ represents the i -th expert network.

Now we introduce the Rollout Routing Replay. Assume that during the inference stage, the input to the MoE layer is $\mathbf{x}_{\text{infer}}$. The router then computes the inference logits $\mathbf{s}_{\text{infer}} = \mathbf{x}_{\text{infer}} \mathbf{W}_r$, from which the routing mask $\mathbf{I}_{\text{infer}} = \text{TopKMask}(\mathbf{s}_{\text{infer}}, K)$ is obtained. The main idea of Rollout Routing Replay is to reuse the inference routing mask $\mathbf{I}_{\text{infer}}$ during the training forward pass while still applying the softmax to the training logits to preserve gradient flow. Specifically, along this replay path, the “replay” gating weights g_{replay} are computed as:

$$g_{\text{replay},i} = \frac{I_{\text{infer},i} \exp(s_{\text{train},i})}{\sum_{j=1}^M I_{\text{infer},j} \exp(s_{\text{train},j})} \quad \text{for } i = 1, \dots, M. \quad (9)$$

These replay weights are then used to combine the training experts’ outputs, producing the replay output y_{replay} :

$$y_{\text{replay}} = \sum_{i=1}^M g_{\text{replay},i} \mathcal{E}_i(\mathbf{x}_{\text{train}}). \quad (10)$$

This design serves two main purposes: (a) **Aligning training and inference:** Using $\mathbf{I}_{\text{infer}}$ ensures that the experts used during training replay match those selected during inference, eliminating mismatch in expert selection. (b) **Preserving the gradient data flow:** By replaying the mask, we can precisely optimize the experts used when inferring this sequence, avoiding the optimization of unused experts, which may introduce gradient noise.

4.2. Router Mask Caching and Multi-Turn Dialogue Support

Many inference engines use a KVCache prefix caching strategy (Zheng et al., 2024; Kwon et al., 2023) to prevent redundant prefill computations on previously seen context, which significantly reduces the total computation in multiple-turn interactions. We observe that the cached router masks share a similar property: for the same prefix tokens, the MoE router should yield identical results. Therefore, routing masks from inference engines $\mathbf{I}_{\text{infer}}$ can be cached along with the prefix KVcache. Concretely, for each layer and token prefix, the corresponding routing masks are stored with KVCache. When the same prefix occurs and hits the cache, the masks can be reused, eliminating the need for recomputation. This allows Rollout Routing Replay to integrate seamlessly with prefix caching mechanisms.

Caching routing masks is especially beneficial in agent scenarios. Many agent tasks, like software engineering (Jimenez et al., 2024) and web browsing (Wei et al., 2025), involve multiple turns of interactions between autoregressive generation and tool calling. To improve efficiency, these processes directly reuse the KVCache from previous turns, so they do not have to regenerate data that’s already been computed. Routing mask caching enables R3 to remain efficient in RL agent tasks without re-prefilling to generate the routing masks, which is crucial for training large-scale, advanced MoE models.

4.3. Empirical Analysis of R3 on Training-Inference Discrepancies

To evaluate the effectiveness of R3 in reducing training-inference discrepancies, we repeat the procedure described in Section 3.1 with Qwen3-30B-A3B model. In this process, we cache the routing distributions obtained during inference on SGLang and replay them within the Megatron framework. After obtaining the inference engine probability and training engine probability, using the equation 3, we estimate the KL

divergence between training and inference. Results shows that after applying R3, the KL divergence between training and inference decrease from 1.5×10^{-3} to 7.5×10^{-4} , which is near the 6.4×10^{-4} observed for the dense model. This intuitively indicates a reduction in the training-inference discrepancy. We also drew the cumulative distribution plot of the ratio of training-inference discrepancies in Fig. 2d with R3. The plot indicates that, for the MoE model, applying R3 reduces by a factor of 1/10 the frequency of tokens with large training-inference discrepancies, reaching a level comparable to the dense model.

5. Experiments

In this section, we evaluate the performance improvement of our R3 method for reinforcement learning and compare it with other baseline methods.

5.1. Setting

Task and Dataset We choose mathematical reasoning as the target task for training. For the training dataset, we collect and filter problems from many open-source datasets, including BigMath (Albalak et al., 2025), ORZ (Hu et al., 2025), and others, yielding approximately 100,000 verifiable math problems. For the evaluation dataset, we adopt AIME24, AIME25, AMC23, and MATH500 (Hendrycks et al., 2021) (level 5) as our benchmark datasets. We report Avg@32 for AIME24 and AIME25, Avg@16 for AMC23, and Avg@4 for MATH500 (level 5). During training, some models may experience performance degradation or even collapse in later stages. To ensure a fair evaluation, we measure model performance every 5 global steps throughout a single training run. We then report the highest observed performance along with the corresponding training step at which it occurs.

Models We select two models for our experiments: (a) Qwen3-30B-A3B-Base (Yang et al., 2025) (b) Qwen3-30B-A3B-SFT, which is fine-tuned from Qwen3-30B-A3B-Base on our general instruction-following dataset.

Baseline Methods We consider the following baseline optimization methods for comparison: (a) **GRPO** (Shao et al., 2024), additionally applies the *Clip Higher* technique from DAPO (Yu et al., 2025), with parameters $\epsilon_{\text{low}} = 0.2$ and $\epsilon_{\text{high}} = 0.27$; (b) **TIS** (Yao et al., 2025), employs rollout importance sampling and uses an upper clipping threshold $C = 2$; (c) **GSPO** (Zheng et al., 2025), employs sequence-level importance sampling, with parameters $\epsilon_{\text{low}} = 3 \times 10^{-4}$ and $\epsilon_{\text{high}} = 4 \times 10^{-4}$. Since our R3 method is orthogonal to optimizers such as GSPO or TIS, we also evaluate various combinations of these techniques.

Multiple Mini Steps vs. Single Mini Step In PPO-like algorithms, a batch of samples collected in one global step is typ-

Table 1. Main evaluation results. This table presents the best scores (with corresponding global step in parentheses) achieved by various methods on evaluation benchmarks under three training configurations. The table also reports the average score (Avg) and the step at which training collapse occurred (Crash Step).

Best Metric(Best Global Step)						
Method	AIME24(↑)	AIME25(↑)	AMC23(↑)	MATH500 Lv5(↑)	Avg(↑)	Crash Step
Qwen3-30B-A3B-SFT, mini_step=8						
GRPO	32.81(65)	20.73(90)	74.84(60)	71.83(100)	48.84(100)	120
GSPO	55.52(165)	38.23(160)	90.16(125)	86.38(125)	66.76(125)	-
GRPO+R3	57.92(180)	38.02(155)	90.16(155)	88.62(170)	68.05(180)	-
GSPO+R3	58.44(160)	39.17(160)	92.50(165)	87.87(165)	69.00(165)	-
Qwen3-30B-A3B-SFT, mini_step=1						
GRPO	49.06(45)	32.08(50)	86.41(55)	83.77(55)	62.23(55)	60
GRPO+TIS	54.90(85)	36.67(90)	88.59(90)	85.63(85)	66.24(90)	105
GRPO+R3	62.92(165)	41.98(170)	93.91(165)	89.93(155)	71.83(165)	-
GRPO+TIS+R3	58.75(180)	41.15(180)	91.87(175)	88.99(180)	70.14(175)	-
Qwen3-30B-A3B-Base, mini_step=1						
GRPO	50.63(100)	32.60(100)	83.13(100)	80.78(90)	61.69(100)	105
GRPO+TIS	56.77(170)	40.10(170)	92.50(165)	89.37(180)	69.22(170)	-
GRPO+R3	60.94(180)	41.35(180)	92.81(175)	89.74(175)	70.73(180)	-
GRPO+TIS+R3	56.67(170)	40.42(170)	93.12(175)	88.99(165)	69.17(175)	-

ically divided into multiple mini steps. However, previous research (Hao et al., 2025) has shown that applying a strictly on-policy strategy with only a single mini step can yield better performance. We investigate both settings. For multiple mini steps, each PPO mini step trains $2048/8 = 256$ samples with 8 optimizer updates and a learning rate of 1×10^{-6} . In the multiple mini-step scenario, for R3 method we replayed the routing both when recomputing the old policy and updating policy. For the single mini step scenario, all 2048 samples are updated once. Due to fewer updates, we increase the learning rate to 3×10^{-6} . We don’t recompute old policy in this scenario.

Other Setting We implement R3 using the VerL (Sheng et al., 2025) framework, use Megatron (Shoeybi et al., 2019) for training, and SGLang (Zheng et al., 2024) for inference. We set the batch size to 256 and $n = 8$, totaling 2048 samples per round. The maximum prompt length is 2048, and the maximum generation length is 30720. The max global step is 180. We adopt Dynamic Sampling (Yu et al., 2025), retaining only partially correct samples during generation until a sufficient batch size is accumulated for training. No auxiliary loss for expert balancing is introduced.

5.2. Experimental Results

The main evaluation results are shown in Table 1. More detailed evaluation results and training logs, including per-evaluation scores as well as training metrics such as entropy, average reward, gradient norm, and training-inference KL divergence, are provided in the appendix A

Overall Performance. R3 achieves better results across dif-

ferent scenarios. In the multi mini-step setting, GRPO+R3 outperforms GSPO by 1.29. Furthermore, combining R3 with GSPO further improves performance by 0.95 points. In the single mini-step setting, R3 outperforms TIS by 5.58 on SFT model and 1.51 on base model. However, combining R3 with TIS does not yield clear gains and may even degrade performance; for instance, in the single mini-step on SFT model, TIS+R3 scores 3.29 points lower than R3 alone. Since R3 already significantly reduces the policy discrepancy between training and inference, the additional correction from TIS offers negligible benefit (Yao et al., 2025).

Training Stability. In the single mini-step setting, three reinforcement learning processes without R3 collapsed during training. To investigate the causes of these collapses, we plotted the estimated training–inference KL divergence and the extreme token distribution function $F(\tau = 2)$ (eq. 4) at each training step (see Fig. 4). We observed that, in each training process, both the KL divergence and $F(\tau = 2)$ values increased over training process. Moreover, collapsed training runs were almost always accompanied by abnormally high KL and $F(\tau = 2)$ values. For instance, in the SFT model trained using GRPO under the single mini-step setting, after 60 global steps, the value of $F(\tau = 2)$ exceeded 0.1. This indicates that for 10% of the tokens, the probability under the training framework differed from that under the inference framework by more than a factor of 2, demonstrating severe training–inference inconsistency. In contrast, for training processes using R3, the value of $F(\tau = 2)$ remained below 10^{-4} . By aligning the training and inference distributions, R3 effectively stabilized

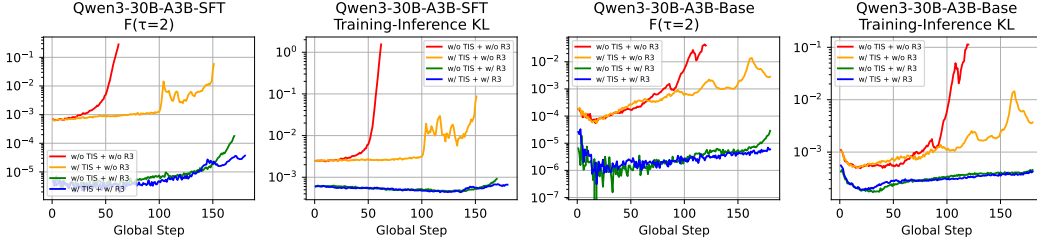


Figure 4. Analysis of training-inference collapse. The plot shows the estimated training-inference KL divergence and the extreme token distribution function $F(\tau = 2)$ (Eq. 4) at each training step.

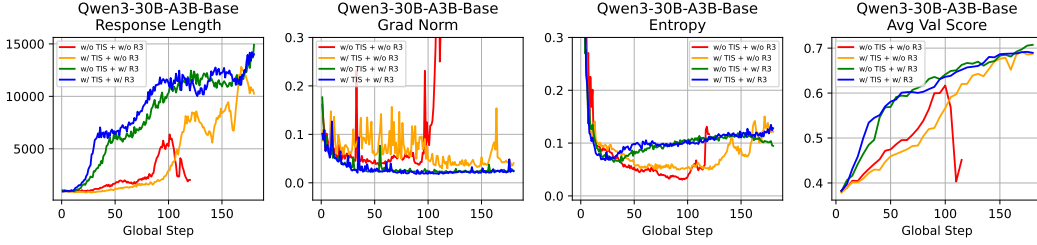


Figure 5. Training dynamics of Qwen3-30B-A3B-Base, including response length, gradient norm, entropy, and average validation score throughout the training process

reinforcement learning for MoE models.

Optimization and Generation Behavior. During training, R3 also enhances optimization stability, exploration behavior, and generation dynamics. We plotted the sequence length, gradient norm, generation entropy, and evaluation score (see Fig. 5). It shows that R3 has smaller gradient norms, a smoother sequence growth pattern, and more stable entropy. (a) **Sequence Length:** With R3, the generated sequence length rises rapidly at the beginning of training, indicating that R3 can quickly capture the correct optimization direction. By contrast, the other two training processes increase only slowly after step 80 and show more pronounced fluctuations. (b) **Gradient Norm:** R3 maintains consistently stable gradient norms, indicating a more stable optimization process. (c) **Generation Entropy:** With R3, entropy begins to increase steadily after about 25 step, suggesting that the model starts exploring better strategies earlier. Without R3, entropy increases much later and fluctuates heavily.

5.3. Generalization

5.3.1. GENERALIZATION TO MULTI-TURN TASK

To verify that R3 can be adapted to multi-turn tasks, we conduct reinforcement learning experiments on the software engineering task. This task has recently become a key benchmark for state-of-the-art models, as it simultaneously evaluates a model’s code generation ability, tool-use capability, and multi-turn reasoning performance. We use the Qwen3-30B-A3B (Yang et al., 2025) model, use R2E-Gym-Lite (Jain et al., 2025) as the training dataset and SWE-bench-Verified (Jimenez et al., 2024) as the test

Table 2. Evaluation results of RL training on SWE Task

Model: Qwen3-30B-A3B		
Method	SWE-bench Verified(↑)	Crash Step
GRPO	31.80	90
GRPO+R3	38.60	-

dataset. Detailed experimental settings are provided in Appendix D. The results are presented in Table 2. The training process of GRPO without R3 collapses after around 90 training steps, while GRPO+R3 maintains stability and achieves higher scores. This result confirms that R3 generalizes well to multi-turn reinforcement learning settings.

5.3.2. GENERALIZATION TO OTHER MOE MODEL

Table 3. Evaluation results of the RL training process of Deepseek-V2-Lite-ReasoningSFT

Model: Deepseek-V2-Lite-ReasoningSFT		
Method	Math Avg(↑)	Crash Step
GRPO	23.55(55)	100
GRPO+R3	26.43(235)	-

To evaluate the performance of R3 on other MoE models, we conduct RL training experiments on Deepseek-V2-Lite (DeepSeek-AI et al., 2024). We first fine-tune Deepseek-V2-Lite on the open-source long reasoning dataset Mixture-of-Thoughts, producing the Deepseek-V2-Lite-ReasoningSFT model. We then conduct reinforcement learning experiments on this model. Detailed experimental settings are

provided in Appendix B.1. The results show that R3 also stabilizes training and achieves higher scores, indicating that R3 generalizes well across different model architectures.

5.4. Speed and Memory Analyze

5.4.1. END-TO-END THROUGHPUT BENCHMARK

To show that R3 can be practically deployed in large-scale training pipelines without incurring prohibitive computational costs. We conduct an end-to-end throughput benchmark to quantify the runtime overhead introduced by R3.

Experiments are performed on the Qwen3-30B-A3B model under two configurations: with and without R3 enabled. Both settings employ identical hardware and software environments to ensure a fair comparison. We set batch size = 256, $n = 8$, and a maximum generation length of 32K. We capture the complete end-to-end execution latency per training step, encompassing inference, forward computation, backward propagation, optimizer update, and all communication overhead between inference and training components.

Table 4. Throughput analysis result of Rollout Routing Replay

Metric	w/o R3	w/ R3
# Global Step	10	10
# Token	203,989,395	203,201,992
Runtime (sec)	4820	4967
# Token/sec	42,321	40,910
Relative Time Cost	100%	103.45%

As summarized in Table 4, enabling R3 results in only a **3.45%** reduction in end-to-end training throughput. This demonstrates that R3 can be efficiently integrated into large-scale reinforcement learning pipelines with negligible impact on training efficiency.

5.4.2. MEMORY USAGE ANALYSIS

R3 only introduces a small additional memory overhead on CPU memory. We keep the router mask in CPU memory most of the time, and only load it onto the GPU when the corresponding sequence is loaded into the training framework. Therefore, storing the router mask only consumes CPU memory. In the Appendix G, we provide a detailed analysis of the memory overhead of training caused by R3. In summary, in our experimental setup, each node consumed an additional 2.5GB of CPU memory during the inference phase and 1.5GB during the training phase.

6. Related Works

6.1. Mixture of Experts

The Mixture-of-Experts (MoE) architecture, originating as an ensemble method to route inputs to specialized subnet-

works (Jacobs et al., 1991; Shazeer et al., 2017), has become a cornerstone for scaling modern language models. By employing a gating network to sparsely activate only a subset of expert parameters per token, MoE decouples a model’s total parameter count from its inference cost, enabling a substantial increase in model capacity. This computational efficiency has driven its adoption in state-of-the-art Transformer models (Jiang et al., 2024; Liu et al., 2024; Team et al., 2025; Yang et al., 2025).

6.2. Reinforcement Learning with Verifiable Rewards

Reinforcement Learning with verifiable rewards (RLVR) has become a standard method for refining the complex reasoning (Xie et al., 2025), mathematical (Shao et al., 2024; Hu et al., 2025), and code abilities (Luo et al., 2025b) of large language models. Algorithms such as GRPO (Shao et al., 2024) DAPO (Yu et al., 2025) and GSPO (Zheng et al., 2025) are now widely adopted. These methods have already been used to train state-of-the-art reasoning models (Yang et al., 2025; Xia et al., 2025; Chen et al., 2025; Guo et al., 2025; Team et al., 2025).

6.3. Nondeterminism, Framework Discrepancies and Reinforcement Learning

With the rise of LLM RL, training-inference discrepancies and non-determinism issues have also drawn widespread attention, largely due to concerns about training stability. Chen et al. (2025) pointed out the discrepancies problem between training and inference can cause instability problems in reinforcement learning and alleviated it by raising the output layer’s precision to FP32. Yuan et al. (2025) examined how numerical precision affects inference nondeterminism. He & Lab (2025) discussed the main reasons why inference can become nondeterministic. From the perspective of importance sampling, Yao et al. (2025), Zhao et al. (2025), and Liu et al. (2025) proposed the TIS, Icepop, and MIS methods to address the problem. Qi et al. (2025) pointed out that using FP16 can resolve the training-inference discrepancies issue.

7. Conclusion

In this work, we identify training-inference routing discrepancies as the primary source of instability in MoE reinforcement learning. To address this, we propose Rollout Routing Replay (R3), which reuses inference-time routing distributions during training to align expert selection while preserving gradient flow. Experiments across multiple RL settings demonstrate that R3 substantially reduces training-inference divergence, stabilizes training, and consistently outperforms existing methods. Our results demonstrate the importance of aligning training and inference in MoE models and show that R3 provides a practical solution for improving stability.

Acknowledgments

This paper is supported by NSFC project 62476009

This work is done during the first author’s internship at Xiaomi

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Albalak, A., Phung, D., Lile, N., Rafailov, R., Gandhi, K., Castricato, L., Singh, A., Blagden, C., Xiang, V., Mahan, D., and Haber, N. Big-math: A large-scale, high-quality math dataset for reinforcement learning in language models, 2025. URL <https://arxiv.org/abs/2502.17387>.
- Bercovich, A., Levy, I., Golan, I., Dabbah, M., El-Yaniv, R., Puny, O., Galil, I., Moshe, Z., Ronen, T., Nabwani, N., Shahaf, I., Tropp, O., Karpas, E., Zilberstein, R., Zeng, J., Singhal, S., Bukharin, A., Zhang, Y., Konuk, T., Shen, G., et al. Llama-nemotron: Efficient reasoning models, 2025. URL <https://arxiv.org/abs/2505.00949>.
- Chen, A., Li, A., Gong, B., Jiang, B., Fei, B., Yang, B., Shan, B., Yu, C., Wang, C., Zhu, C., et al. Minimax-m1: Scaling test-time compute efficiently with lightning attention. *arXiv preprint arXiv:2506.13585*, 2025.
- DeepSeek-AI, Liu, A., Feng, B., Wang, B., Wang, B., Liu, B., Zhao, C., Deng, C., Ruan, C., Dai, D., Guo, D., et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model, 2024. URL <https://arxiv.org/abs/2405.04434>.
- Face, H. Open r1: A fully open reproduction of deepseek-r1, January 2025. URL <https://github.com/huggingface/open-r1>.
- Guo, D., Yang, D., Zhang, H., Song, J., Wang, P., Zhu, Q., Xu, R., Zhang, R., Ma, S., Bi, X., Zhang, X., Yu, X., Wu, Y., Wu, Z. F., Gou, Z., Shao, Z., Li, Z., Gao, Z., Liu, A., Xue, B., Wang, B., Wu, B., Feng, B., Lu, C., Zhao, C., Deng, C., Ruan, C., Dai, D., Chen, D., Ji, D., Li, E., Lin, F., Dai, F., Luo, F., Hao, G., Chen, G., Li, G., Zhang, H., Xu, H., Ding, H., Gao, H., Qu, H., Li, H., Guo, J., Li, J., Chen, J., Yuan, J., Tu, J., Qiu, J., Li, J., Cai, J. L., Ni, J., Liang, J., Chen, J., Dong, K., Hu, K., You, K., Gao, K., Guan, K., Huang, K., Yu, K., Wang, L., Zhang, L., Zhao, L., Wang, L., Zhang, L., Xu, L., Xia, L., Zhang, M., Zhang, M., Tang, M., Zhou, M., Li, M., Wang, M., Li, M., Tian, N., Huang, P., Zhang, P., Wang, Q., Chen, Q., Du, Q., Ge, R., Zhang, R., Pan, R., Wang, R., Chen, R. J., Jin, R. L., Chen, R., Lu, S., Zhou, S., Chen, S., Ye, S., Wang, S., Yu, S., Zhou, S., Pan, S., Li, S. S., Zhou, S., Wu, S., Yun, T., Pei, T., Sun, T., Wang, T., Zeng, W., Liu, W., Liang, W., Gao, W., Yu, W., Zhang, W., Xiao, W. L., An, W., Liu, X., Wang, X., Chen, X., Nie, X., Cheng, X., Liu, X., Xie, X., Liu, X., Yang, X., Li, X., Su, X., Lin, X., Li, X. Q., Jin, X., Shen, X., Chen, X., Sun, X., Wang, X., Song, X., Zhou, X., Wang, X., Shan, X., Li, Y. K., Wang, Y. Q., Wei, Y. X., Zhang, Y., Xu, Y., Li, Y., Zhao, Y., Sun, Y., Wang, Y., Yu, Y., Zhang, Y., Shi, Y., Xiong, Y., He, Y., Piao, Y., Wang, Y., Tan, Y., Ma, Y., Liu, Y., Guo, Y., Ou, Y., Wang, Y., Gong, Y., Zou, Y., He, Y., Xiong, Y., Luo, Y., You, Y., Liu, Y., Zhou, Y., Zhu, Y. X., Huang, Y., Li, Y., Zheng, Y., Zhu, Y., Ma, Y., Tang, Y., Zha, Y., Yan, Y., Ren, Z. Z., Ren, Z., Sha, Z., Fu, Z., Xu, Z., Xie, Z., Zhang, Z., Hao, Z., Ma, Z., Yan, Z., Wu, Z., Gu, Z., Zhu, Z., Liu, Z., Li, Z., Xie, Z., Song, Z., Pan, Z., Huang, Z., Xu, Z., Zhang, Z., and Zhang, Z. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, September 2025. ISSN 1476-4687. doi: 10.1038/s41586-025-09422-z. URL <http://dx.doi.org/10.1038/s41586-025-09422-z>.
- Hao, Y., Dong, L., Wu, X., Huang, S., Chi, Z., and Wei, F. On-policy rl with optimal reward baseline, 2025. URL <https://arxiv.org/abs/2505.23585>.
- He, H. and Lab, T. M. Defeating nondeterminism in llm inference. *Thinking Machines Lab: Connectionism*, 2025. doi: 10.64434/tml.20250910. <https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/>.
- Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. Measuring mathematical problem solving with the MATH dataset. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*, 2021. URL <https://openreview.net/forum?id=7Bywt2mQsCe>.
- Hu, J., Zhang, Y., Han, Q., Jiang, D., Zhang, X., and Shum, H.-Y. Open-reasoner-zero: An open source approach to scaling up reinforcement learning on the base model, 2025. URL <https://arxiv.org/abs/2503.24290>.
- Jacobs, R. A., Jordan, M. I., Nowlan, S. J., and Hinton, G. E. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 03 1991. ISSN 0899-7667. doi: 10.1162/neco.1991.3.1.79. URL <https://doi.org/10.1162/neco.1991.3.1.79>.
- Jain, N., Singh, J., Shetty, M., Zhang, T., Zheng, L., Sen, K., and Stoica, I. R2e-gym: Procedural environment genera-

- tion and hybrid verifiers for scaling open-weights SWE agents. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=7evvwdo3z>.
- Jiang, A. Q., Sablayrolles, A., Roux, A., Mensch, A., Savary, B., Bamford, C., Chaplot, D. S., Casas, D. d. l., Hanna, E. B., Bressand, F., et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. R. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VTF8yNQm66>.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP '23*, pp. 611–626, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400702297. doi: 10.1145/3600006.3613165. URL <https://doi.org/10.1145/3600006.3613165>.
- Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- Liu, J., Li, Y., Fu, Y., Wang, J., Liu, Q., and Shen, Y. When speed kills stability: Demystifying RL collapse from the training-inference mismatch, September 2025. URL <https://richardli.xyz/rl-collapse>.
- Lozhkov, A., Kydlíček, H., Allal, L. B., Penedo, G., Beeching, E., Gallouédec, Q., Habib, N., Tunstall, L., and von Werra, L. Openrl-math-220k. <https://huggingface.co/datasets/open-rl/OpenR1-Math-220k>, 2025.
- Luo, M., Jain, N., Singh, J., Tan, S., Patel, A., Wu, Q., Ariyak, A., Cai, C., Venkat, T., Zhu, S., Athiwaratkun, B., Roongta, M., Zhang, C., Li, L. E., Popa, R. A., Sen, K., and Stoica, I. Deepswe: Training a state-of-the-art coding agent from scratch by scaling rl. <https://pretty-radio-b75.notion.site/DeepSWE-Training-a-Fully-Open-sourced-State-of-the-Art-Coding-Agent-by-Scaling-RL-22281902c1468193aabb9a8c59bbe33>, 2025a. Notion Blog.
- Luo, M., Tan, S., Huang, R., Patel, A., Ariyak, A., Wu, Q., Shi, X., Xin, R., Cai, C., Weber, M., Zhang, C., Li, L. E., Popa, R. A., and Stoica, I. Deepcoder: A fully open-source 14b coder at o3-mini level. <https://pretty-radio-b75.notion.site/DeepCoder-A-Fully-Open-Source-14B-Coder-at-O3-mini-Level-1cf81902c14680b3bee5eb349a512a51>, 2025b. Notion Blog.
- OpenAI. Learning to reason with llms, 2024. URL <https://openai.com/index/learning-to-reason-with-llms/>.
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P. F., Leike, J., and Lowe, R. Training language models to follow instructions with human feedback. In Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., and Oh, A. (eds.), *Advances in Neural Information Processing Systems*, volume 35, pp. 27730–27744. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/b1efde53be364a73914f58805a001731-Paper-Conference.pdf.
- Penedo, G., Lozhkov, A., Kydlíček, H., Allal, L. B., Beeching, E., Lajarín, A. P., Gallouédec, Q., Habib, N., Tunstall, L., and von Werra, L. Codeforces cots. <https://huggingface.co/datasets/open-rl/codeforces-cots>, 2025.
- Qi, P., Liu, Z., Zhou, X., Pang, T., Du, C., Lee, W. S., and Lin, M. Defeating the training-inference mismatch via fp16, 2025. URL <https://arxiv.org/abs/2510.26788>.
- Schulman, J. Approximating kl divergence, 2020. URL <http://joschu.net/blog/kl-approx.html>.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Zhang, M., Li, Y., Wu, Y., and Guo, D. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017. URL <https://openreview.net/forum?id=BlckMDqlg>.
- Sheng, G., Zhang, C., Ye, Z., Wu, X., Zhang, W., Zhang, R., Peng, Y., Lin, H., and Wu, C. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems, EuroSys '25*,

- pp. 1279–1297, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400711961. doi: 10.1145/3689031.3696075. URL <https://doi.org/10.1145/3689031.3696075>.
- Shoeybi, M., Patwary, M., Puri, R., LeGresley, P., Casper, J., and Catanzaro, B. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- Team, K., Du, A., Gao, B., Xing, B., Jiang, C., Chen, C., Li, C., Xiao, C., Du, C., Liao, C., et al. Kimi k1. 5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599*, 2025.
- Wei, J., Sun, Z., Papay, S., McKinney, S., Han, J., Fulford, I., Chung, H. W., Passos, A. T., Fedus, W., and Glaese, A. Browsecomp: A simple yet challenging benchmark for browsing agents, 2025. URL <https://arxiv.org/abs/2504.12516>.
- Xia, B., Shen, B., Cici, Zhu, D., Zhang, D., Wang, G., Zhang, H., Liu, H., Xiao, J., Dong, J., Zhao, L., Li, P., Wang, P., Yu, S., Chen, S., Wang, W., Ma, W., Deng, X., Huang, Y., Song, Y., Jiang, Z., Ye, B., Cai, C., He, C., Zhang, D., Zhang, D., Wang, G., Tian, H., Zhao, H., Qu, H., Xu, H., Shi, J., Bao, K., Fang, K., Zhou, K., Zhou, K., Li, L., Zhu, M., Chen, N., Wang, Q., Liu, S., Li, S., Gu, S., Ren, S., Liu, S., Deng, S., Zhuang, W., Lv, W., Yang, W., Zhang, X., Yong, X., Zhang, X., Song, X., Xu, X., Wang, X., Yan, Y., Tu, Y., Tian, Y., Wang, Y., Yu, Y., Lin, Z., Song, Z., and Yue, Z. Mimo: Unlocking the reasoning potential of language model – from pretraining to posttraining, 2025. URL <https://arxiv.org/abs/2505.07608>.
- Xie, T., Gao, Z., Ren, Q., Luo, H., Hong, Y., Dai, B., Zhou, J., Qiu, K., Wu, Z., and Luo, C. Logic-rl: Unleashing llm reasoning with rule-based reinforcement learning. *arXiv preprint arXiv:2502.14768*, 2025.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Zhou, J., Lin, J., Dang, K., Bao, K., Yang, K., Yu, L., Deng, L., Li, M., Xue, M., Li, M., Zhang, P., Wang, P., Zhu, Q., Men, R., Gao, R., Liu, S., Luo, S., Li, T., Tang, T., Yin, W., Ren, X., Wang, X., Zhang, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Zhang, Y., Wan, Y., Liu, Y., Wang, Z., Cui, Z., Zhang, Z., Zhou, Z., and Qiu, Z. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Yao, F., Liu, L., Zhang, D., Dong, C., Shang, J., and Gao, J. Your efficient rl framework secretly brings you off-policy rl training, August 2025. URL <https://fengyao.notion.site/off-policy-rl>.
- Yu, Q., Zhang, Z., Zhu, R., Yuan, Y., Zuo, X., Yue, Y., Fan, T., Liu, G., Liu, L., Liu, X., et al. Dapo: An open-source llm reinforcement learning system at scale, 2025. URL <https://arxiv.org/abs/2503.14476>, 2025.
- Yuan, J., Li, H., Ding, X., Xie, W., Li, Y.-J., Zhao, W., Wan, K., Shi, J., Hu, X., and Liu, Z. Understanding and mitigating numerical sources of nondeterminism in LLM inference. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=Q3qASZAEZw>.
- Zhao, X., Liu, Y., Xu, K., Guo, J., Wang, Z., Sun, Y., Kong, X., Cao, Q., Jiang, L., Wen, Z., Zhang, Z., and Zhou, J. Small leak can sink a great ship–boost rl training on moe with icepop!, Sep 2025. URL <https://ringtech.notion.site/icepop>.
- Zheng, C., Liu, S., Li, M., Chen, X.-H., Yu, B., Gao, C., Dang, K., Liu, Y., Men, R., Yang, A., Zhou, J., and Lin, J. Group sequence policy optimization, 2025. URL <https://arxiv.org/abs/2507.18071>.
- Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., and Sheng, Y. Sglang: Efficient execution of structured language model programs. In Globerson, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J., and Zhang, C. (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 62557–62583. Curran Associates, Inc., 2024. doi: 10.52202/079017-2000. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/724be4472168f31balc9ac630f15dec8-Paper-Conference.pdf.

A. Detailed Evaluation Results and Training Metrics

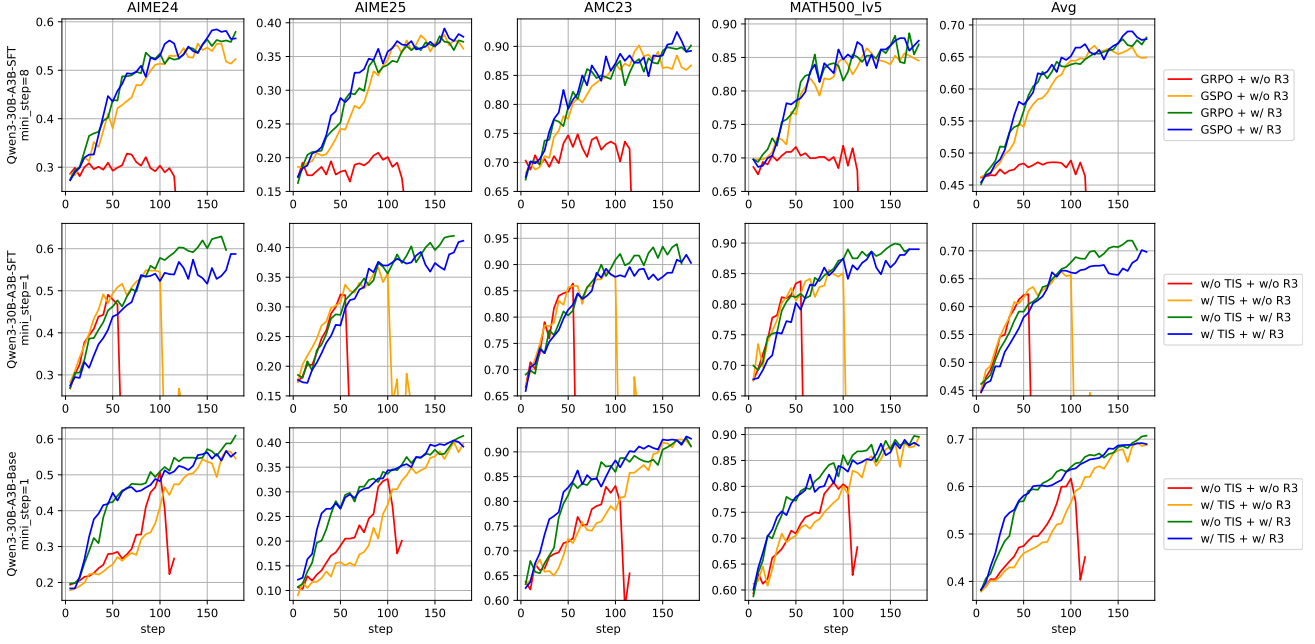


Figure 6. The detailed evaluation results of the experiment in Section 5.

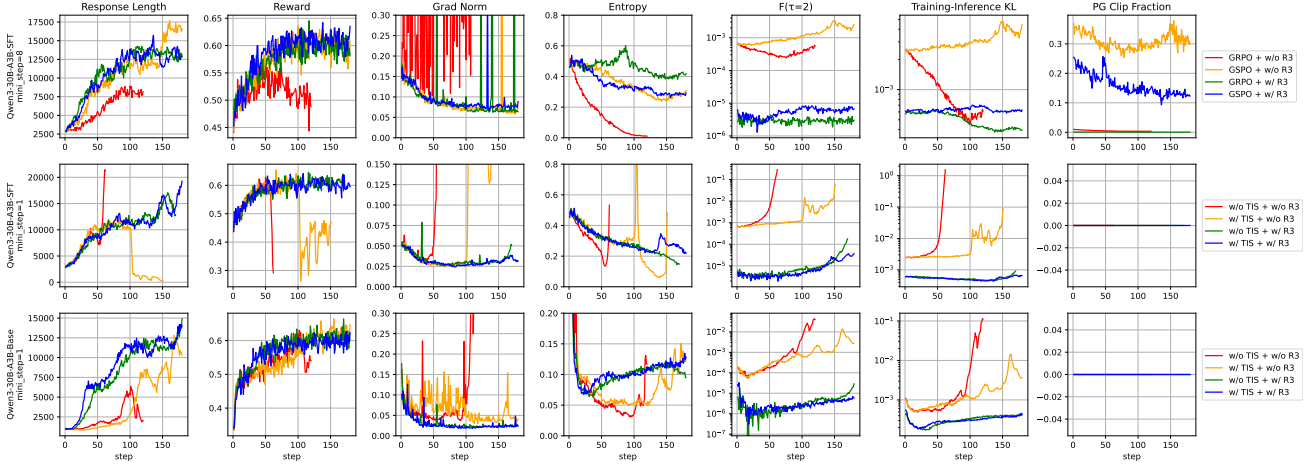


Figure 7. The detailed training metrics of the experiment in Section 5.

B. Other Model Experiments

B.1. Experiments on Deepseek-V2-Lite

To evaluate the performance of R3 on other MoE models, we conduct experiments on Deepseek-V2-Lite (DeepSeek-AI et al., 2024). We first fine-tune Deepseek-V2-Lite on the open-source long reasoning dataset Mixture-of-Thoughts¹ (Face, 2025; Penedo et al., 2025; Lozhkov et al., 2025; Bercovich et al., 2025), producing the Deepseek-V2-Lite-ReasoningSFT model. We then conduct reinforcement learning experiments on this model.

We compare GRPO and GRPO+R3 with a learning rate of 3×10^{-6} when `mini_step` = 1; all other hyperparameters follow the configuration described in Section 5.1.

The results are presented in Table 5. The experiments show that training runs without R3 eventually collapse (for example, when `mini_step` = 1, GRPO collapses at training step 100), whereas training runs with R3 remain stable for up to 250 training steps. These findings demonstrate that the collapse issue occurs not only in Qwen3 MoE but also in other MoE models, and that R3 can likewise ensure stable training across these settings.

Table 5. Evaluation results of the RL training process of Deepseek-V2-Lite-ReasoningSFT

Best Metric(Best Global Step)						
Method	AIME24(↑)	AIME25(↑)	AMC23(↑)	MATH500 Lv5(↑)	Avg(↑)	Crash Step
Deepseek-V2-Lite-ReasoningSFT, mini_step=1, max_global_step=250						
GRPO	6.87(60)	9.69(55)	42.50(50)	37.50(55)	23.55(55)	100
GRPO+R3	8.85 (210)	11.46 (145)	48.28 (165)	43.28 (235)	26.43 (235)	-

B.2. Experiments on Qwen3-8B-Base

In Section 5.1, we noted that dense models, which lack a router, should naturally exhibit greater stability than MoE models. To verify this point, we conduct reinforcement learning experiments on Qwen3-8B-Base following the same experimental setup as in Section 5.1. Since dense models do not contain a router, R3 cannot be applied, and thus we only perform the baseline experiments.

The results, shown in Table 6, indicate that the Qwen3-8B-Base model maintains stable training for up to 180 steps. Combined with the findings in Section 5, this demonstrates that dense models inherently possess better stability than MoE models, further confirming that router behavior is a key factor underlying the instability of reinforcement learning in MoE architectures.

Table 6. Evaluation results of the RL training process of Qwen3-8B-Base

Final Metric						
Method	AIME24(↑)	AIME25(↑)	AMC23(↑)	MATH500 Lv5(↑)	Avg(↑)	Crash Step
Qwen3-8B-Base, mini_step=1, max_global_step=180						
GRPO	41.35	29.06	84.53	82.65	59.40	-

B.3. Training-Inference Discrepancy Analyze on Additional Models

We evaluate $F(\tau) = 2$ and the KL divergence for several models with and without R3, including Qwen3-30B-A3B, Deepseek-V2-Lite, Mixtral-8x7B, and Moonlight-16B-A3B. The results are as follows. "SFT" indicates that the model has been fine-tuned.

From the results, it can be seen that R3 can improve training and inference consistency in many types of MoE models. This includes models with different router activations (Qwen3-30B-A3B, DeepSeek-V2-Lite, and Mixtral-8x7B use softmax, while Moonlight-16B-A3B uses sigmoid), different numbers of experts (DeepSeek-V2-Lite and Moonlight-16B-A3B have 64 experts, Qwen3-30B-A3B has 128 experts, and Mixtral has only 8 experts), and whether shared experts are used

¹<https://huggingface.co/datasets/open-r1/Mixture-of-Thoughts>

Table 7. Training-Inference Discrepancy

Model & Method	$F(\tau) = 2$	KL
Qwen3-30B-A3B	2.54e-4	1.37e-3
Qwen3-30B-A3B + R3	5.83e-6	7.03e-4
Deepseek-V2-Lite(SFT)	2.25e-3	4.06e-3
Deepseek-V2-Lite(SFT) + R3	2.16e-4	1.17e-3
Mixtral-8x7B(SFT)	9.77e-5	1.03e-3
Mixtral-8x7B(SFT) + R3	1.08e-6	4.98e-4
Moonlight-16B-A3B-Instruct	1.73e-5	3.67e-4
Moonlight-16B-A3B-Instruct + R3	6.94e-7	1.92e-4

(Moonlight-16B-A3B and DeepSeek-V2-Lite use shared experts). This shows the general applicability of the R3 method.

C. Fix Router Experiments

We repeated a controlled study on the Qwen3-30B-A3B-Base model to investigate whether freezing the router alone prevents training collapse. In this experiment the router parameters were frozen, while all other training hyperparameters and procedures strictly followed the configuration given in Section 5.1. The results are shown in Table 8

We observe that freezing the router does not prevent catastrophic collapse: the model still experiences a pronounced deterioration in performance at approximately `training_step = 120`. The experiment implies that attributing collapse solely to failures in the router parameters is insufficient. Instead, it suggests that the collapse phenomenon involves degradation of the model parameters as a whole rather than an isolated malfunction of the router.

Table 8. Evaluation results of the RL training process of fixed router

Method	Best Metric(Best Global Step)					
	AIME24(↑)	AIME25(↑)	AMC23(↑)	MATH500 Lv5(↑)	Avg(↑)	Crash Step
Qwen3-30B-A3B-Base, mini_step=1, max_global_step=180						
GRPO+FixRouter	49.27(115)	33.75(115)	84.38(115)	82.46(115)	62.47(115)	120
GRPO+R3	60.94 (180)	41.35 (180)	92.81 (175)	89.74 (175)	70.73 (180)	-

D. Multi-Turn Task Reinforcement Learning Experiments

In Section 4.2, we mentioned that the R3 method can be adapted to Multi-Turn tasks. To verify this capability, we conduct reinforcement learning experiments on the software engineering (Jimenez et al., 2024) task using the Qwen3-30B-A3B (Yang et al., 2025) model. We use R2E-Gym-Lite² (Jain et al., 2025) as the training dataset and SWE-bench Verified³ (Jimenez et al., 2024) as the validation dataset. For training, We set the maximum sequence length to 65536 tokens (including the prompt, model response, and environment observation), the batch size to 64, the maximum number of interaction steps to 50, the learning rate to 2×10^{-6} , and the mini_step to 1. For evaluation, We report Pass@1 on SWE-bench Verified. We evaluate the model performance every 10 global steps. We compare the performance of GRPO and GRPO+R3 under these settings. All other hyperparameters follow the configuration described in Section 5.1.

The results are presented in Table 9, and the detailed training metrics are provided in Figure 8. The training process of GRPO without R3 collapses after around 90 training steps, while GRPO+R3 maintains stable. The final performance of GRPO+R3 reaches a Pass@1 of 38.6, which is 6.8 points higher than that of GRPO. Since we use the Router Mask Caching technique described in Section 4.2, we do not need to re-prefill the prefix to obtain the routing mask, thus the rollout speed is not slowed down. These results confirm that R3 generalizes well to multi-turn reinforcement learning settings.

Table 9. Evaluation results of RL training on SWE Task with Qwen3-30B-A3B

Qwen3-30B-A3B, mini_step=1, max_global_step=180			
Method	SWE-bench Verified(↑)	Best Global Step	Crash Step
GRPO	31.80	70	90
GRPO+R3	38.60	160	-

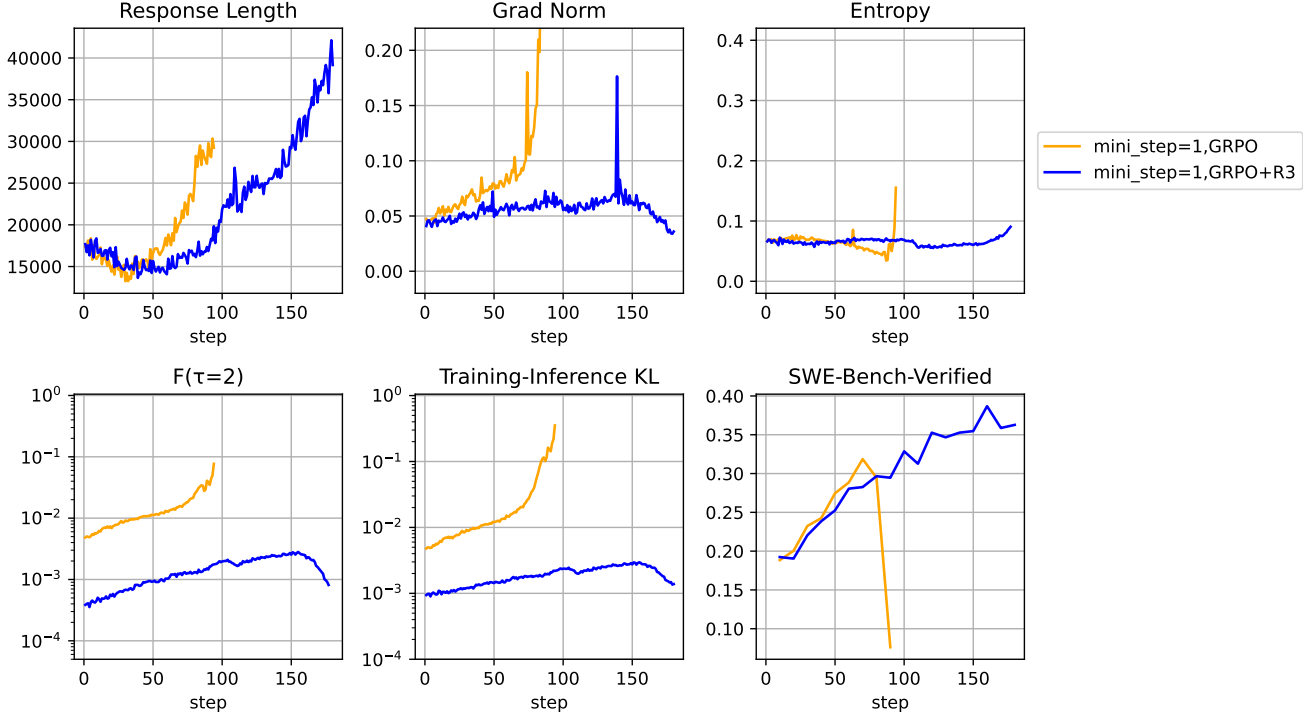


Figure 8. The detailed training and evaluation results of the SWE experiment.

²<https://huggingface.co/datasets/R2E-Gym/R2E-Gym-Lite>

³https://huggingface.co/datasets/princeton-nlp/SWE-bench_Verified

E. Additional Experiments of Reasoning SFT Model

To evaluate the performance of R3 on reasoning SFT models, we fine-tune Qwen3-30B-A3B-Base (Yang et al., 2025) on the open-source long reasoning dataset Mixture-of-Thoughts (Face, 2025; Penedo et al., 2025; Lozhkov et al., 2025; Bercovich et al., 2025), producing the Qwen3-30B-A3B-ReasoningSFT model. We conduct reinforcement learning experiments on this model. We compare GSPO and GRPO+R3 with a learning rate of 2×10^{-6} when `mini_step` = 4, and compare GRPO and GRPO+R3 with a learning rate of 3×10^{-6} when `mini_step` = 1; all other hyperparameters follow the configuration described in Section 5.1.

The results are shown in Table 10, and training and validation metrics are illustrated in Fig 9. The experiments show that training processes without R3 collapse (for `mini_step` = 4, GSPO collapses at training step 90, and for `mini_step` = 1, GRPO collapses at training step 40), while training processes with R3 remain robust. In addition, training with R3 exhibits a more stable pattern of sequence-length growth as well as more stable entropy and gradient norms throughout the training process.

Table 10. Evaluation results of the RL training process of Qwen3-30B-A3B-ReasoningSFT

Best Metric(Best Global Step)						
Method	AIME24(↑)	AIME25(↑)	AMC23(↑)	MATH500 Lv5(↑)	Avg(↑)	Crash Step
Qwen3-30B-A3B-ReasoningSFT, mini_step=4, max_global_step=180						
GSPO	67.50(35)	48.33(20)	94.21(60)	91.79(35)	74.62(55)	90
GRPO+R3	69.06 (165)	51.66 (155)	95.15 (160)	93.28 (145)	77.00 (165)	-
Qwen3-30B-A3B-ReasoningSFT, mini_step=1, max_global_step=180						
GRPO	68.54(25)	48.95(25)	93.90(30)	91.23(20)	75.30(25)	40
GRPO+R3	69.47 (85)	53.12 (180)	95.46 (170)	93.28 (180)	76.79 (180)	-

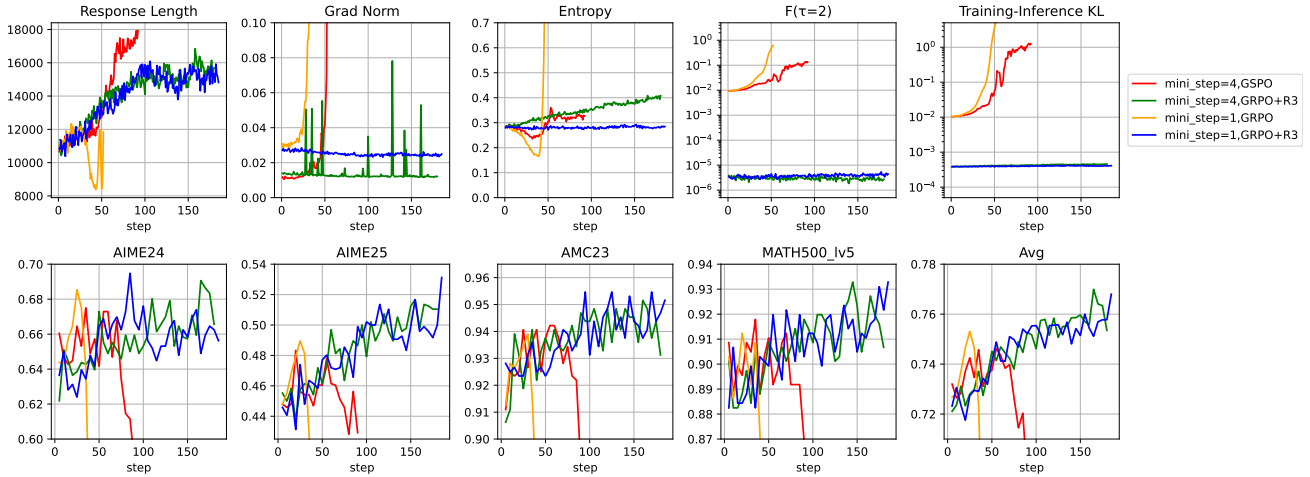


Figure 9. The detailed training metrics and evaluation results of the RL training process of Qwen3-30B-A3B-ReasoningSFT

F. Random Seed Experiments

We have conducted three runs under different seeds for GSPO, TIS on Qwen3-30B-A3B-Base model with mini-step=4, learning rate=2e-6. The results are as follows:

Table 11. Evaluation results of the random seed experiments

Best Metric(Best Global Step)							
Method	Seed	AIME24(↑)	AIME25(↑)	AMC23(↑)	MATH500 Lv5(↑)	Avg(↑)	Mean±Std
Qwen3-30B-A3B-Base, mini_step=4, max_global_step=200							
GSPO	2025	56.35	36.56	87.13	90.47	67.63	65.35±3.40
GSPO	2026	55.73	34.38	87.50	90.31	66.98	
GSPO	2027	45.21	35.52	80.04	85.00	61.44	
TIS	2025	47.81	33.13	84.89	86.41	63.06	63.82±1.16
TIS	2026	49.58	32.50	84.33	86.56	63.24	
TIS	2027	51.56	35.00	85.82	88.28	65.16	
R3	2025	52.81	37.92	84.14	88.13	65.75	67.39±1.73
R3	2026	57.54	39.06	88.81	91.41	69.20	
R3	2027	56.04	35.31	88.43	89.06	67.21	

From the results, we can see that R3 achieved the best performance. GSPO shows large performance fluctuations between runs and can easily produce bad cases, while TIS performs slightly worse.

G. Routing-Mask Caching Size Analysis

G.1. Routing-Mask Cache in Inference Framework Compared with KV Cache

For the inference framework, the routing-mask cache is stored in CPU memory and is only $\frac{1}{128}$ the size of the KV cache stored in GPU HBM. Routing masks are maintained alongside KV cache entries, but with a significantly smaller memory footprint due to their lower dimensionality and their storage on CPU memory. The datatype of the KV cache is `bfloat16` (2 bytes), and the datatype of the routing mask is `int16` (2 bytes).

Assuming 40 GB of GPU memory allocated for KV cache prefix caching per GPU, only about 2.5 GB of CPU memory per node is required to store routing masks for eight GPUs, which results in negligible deployment overhead. The comparison is summarized in Table 12.

Table 12. Comparison of KV Cache and Routing Mask Cache in the Inference Framework

	KV Cache	Routing Mask Cache
Each Token	DataType × #Layer × (KDim + VDim)	DataType × #Layer × TopK
Each Token in Qwen3-30B-A3B	$2 \times 48 \times (512 + 512)$	$2 \times 48 \times 8$
437K Tokens per GPU	40 GB on GPU HBM	0.3 GB on CPU Memory
437K × 8 Tokens per Node (8 GPUs)	40 GB × 8 on HBM	2.5 GB on CPU Memory

G.2. Routing Mask in Training Framework per Global Training Step

For the training framework, the maximum memory footprint of routing masks per global step is determined by the batch configuration. Let the number of prompts be B , the number of groups per prompt be G , the maximum sequence length be N , the number of layers be L , and each layer selects K experts. The routing mask stores the expert indices of all selected experts using the `int16` datatype. The maximum routing-mask memory requirement is

$$\text{Mask Size} = 2 \times B \times G \times N \times L \times K \text{ bytes.}$$

For the configuration $B = 256$, $G = 8$, $N = 32768$, $L = 48$, and $K = 8$, the maximum routing-mask memory footprint is approximately 48 GB.

In practice, these masks are distributed across the training infrastructure rather than being stored on a single machine. They are typically stored in CPU memory and only transferred into GPU memory when required during training.

If the router states are distributed across T training nodes—for example, $T = 32$ —each machine stores only around 1.5 GB of mask data. For modern AI clusters equipped with high-bandwidth interconnects and large memory capacity, both the storage and communication overhead introduced by such routing masks are well within operational limits.

H. Formal Definition of Router-Discrepancy Metrics

As analyzed in Section 3.2, we have already discussed the qualitative differences in routing behavior. Below, we formally define the meanings of the x-axis and y-axis in Figure 3 using mathematical notation.

Let $D = \{s_1, s_2, \dots, s_B\}$ denote a batch of sequences. For each sequence s_i , let its length be N_i , and let the model depth be L . Define the router difference during the forward pass as

$$d(i, t, l) = \text{difference between router selection during training and inference at layer } l \text{ for token } s_{i,t}.$$

Router-Level: (Figure 3(a))

$$y = \frac{1}{L \cdot \sum_{i=1}^B N_i} \sum_{i=1}^B \sum_{t=1}^{N_i} \sum_{l=1}^L \mathbf{1}[d(i, t, l) = x].$$

Token-Level: (Figure 3(b))

$$y = \frac{1}{\sum_{i=1}^B N_i} \sum_{i=1}^B \sum_{t=1}^{N_i} \mathbf{1} \left[\sum_{l=1}^L d(i, t, l) = x \right].$$

Sequence-level: (Figure 3(c))

$$y = \frac{1}{B} \sum_{i=1}^B \mathbf{1} \left[x \leq \frac{1}{N_i} \sum_{t=1}^{N_i} \sum_{l=1}^L d(i, t, l) < x + 1 \right].$$

I. Algorithm Details

We present the algorithmic pseudocode of R3 for both the multi mini-step setting (Algorithm 1) and the single mini-step on-policy setting (Algorithm 2). Note that for clarity, we only highlight the routing replay behavior during the recompute and update stages, omitting many implementation details such as parallel rollout and Dynamic Sampling.

Algorithm 1 GRPO Training with R3

Input: Prompt batch $\mathcal{B}$, model parameters θ , number of mini-steps E

for each global step **do**

// Stage 1: Inference engine rollout and reward

for each prompt $x \in \mathcal{B}$ **do**

 Generate response $y \sim \mu_{\theta_{\text{old}}}(\cdot|x)$ via inference engine

 Cache per-layer routing masks $\{\mathbf{I}_{\text{infer}}^{(l)}\}$ for all tokens

end for

 Obtain reward $r(x, y)$ for each response via verifier

 Compute GRPO advantage $\hat{A}$ from group rewards

// Stage 2: Training engine — recompute old log-probs with R3

 Forward pass replaying $\{\mathbf{I}_{\text{infer}}^{(l)}\}$ to obtain $\pi_{\theta_{\text{old}}}(y_t|x, y_{<t})$

// Stage 3: Mini-step policy optimization with R3

for $e = 1$ **to** E **do**

 Sample mini-batch from $\mathcal{B}$

 Forward pass replaying $\{\mathbf{I}_{\text{infer}}^{(l)}\}$ to obtain $\pi_{\theta}(y_t|x, y_{<t})$

 Compute loss $\mathcal{L}$ using π_{θ} , $\pi_{\theta_{\text{old}}}$, $\mu_{\theta_{\text{old}}}$, and $\hat{A}$

 Backward and update θ

end for

end for

Algorithm 2 On-Policy GRPO Training with R3

Input: Prompt batch $\mathcal{B}$, model parameters θ

for each global step **do**

// Stage 1: Inference engine rollout and reward

for each prompt $x \in \mathcal{B}$ **do**

 Generate response $y \sim \mu_{\theta_{\text{old}}}(\cdot|x)$ via inference engine

 Cache per-layer routing masks $\{\mathbf{I}_{\text{infer}}^{(l)}\}$ for all tokens

end for

 Obtain reward $r(x, y)$ for each response via verifier

 Compute GRPO advantage $\hat{A}$ from group rewards

// Stage 2: Training engine — policy optimization with R3

 Forward pass replaying $\{\mathbf{I}_{\text{infer}}^{(l)}\}$ to obtain $\pi_{\theta}(y_t|x, y_{<t})$

 Compute loss $\mathcal{L}$ using π_{θ} , $\mu_{\theta_{\text{old}}}$, and $\hat{A}$

 Backward and update θ

end for

J. Dataset Filtering Configuration

We filtered the math-problem dataset through multiple stages to construct a high-quality subset for programmatic verification and training.

- **Weak-Model Filtering.** We applied DeepSeek-R1-Distilled-Qwen-7B (Guo et al., 2025) to perform 16 rounds of filtering. Problems with a pass rate greater than 90% under this weak-model evaluation were removed.
- **Strong-Model Filtering.** We applied the stronger DeepSeek-R1-0528 (Guo et al., 2025) models for one round of filtering and collected problems that the strong model could not solve.
- **Problem Rewriting.** For problems that failed the strong-model filtering step, we suspect that some failures are due to formatting issues or proof-style requirements. We therefore used several state-of-the-art models (for example, DeepSeek-V3)(Liu et al., 2024) to attempt automatic rewriting of these problems into forms that can be programmatically verified.
- **Refiltering.** Problems that were rewritten in the previous step were re-evaluated using both the weak-model and strong-model filtering procedures; problems that then passed the filters were removed from the failure set.

K. Problem Verifier Configuration

The verifier is designed to extract a final parseable mathematical expression from the model output and compare it to the reference solution, supporting both purely numeric answers and symbolic expressions.

- **Output Format.** The model is prompted to perform step-by-step reasoning and to place the final answer inside a boxed expression. The prompt template (presented here as a Python f-string) is:

```
f"{problem}\nPlease reason step by step,  
and put your final answer within \boxed{}."
```

- **Verification Tool.** We use the open-source Math-Verify library⁴ to extract the last parseable mathematical expression from the model’s answer and compare it against the reference solution. This verifier configuration supports both pure numeric answers and expressions containing variables by comparing semantic mathematical expressions rather than performing raw string matching.

L. Token Type analysis of routing discrepancies

We conducted an additional analysis focused on quantifying the routing discrepancy at the token level.

Specifically, we evaluated the Qwen3-30B-A3B model on a set of 2,048 mathematical problems. During inference, we recorded the routing distribution for each generated token. We then fed the same token sequences through the training framework in a forward pass (without R3), again collecting routing distribution for every token.

For each token, we computed its routing discrepancy, defined as the difference in selected experts between the two passes. The discrepancy ranges from 0 to $L \times K$, where L denotes the number of MoE layers and K represents the number of activated experts. Afterward, we filtered out low-frequency tokens from vocabulary (frequency threshold: 0.02%) and computed the mean discrepancy for the remaining tokens. The top 50 tokens with the highest mean discrepancy are shown in Table 13.

In Figure 10, we visualize the token-level routing discrepancy of a response segment, with red indicating greater routing differences.

A clear pattern emerges: tokens corresponding to mathematical computation exhibit substantially higher routing discrepancy compared to others.

⁴<https://github.com/huggingface/Math-Verify>

One possible explanation is that performing mathematical computation requires the model to simulate complex reasoning steps in the hidden space. During this process, the hidden states change more sharply, which makes the routing decisions less stable and leads to larger differences between training and inference.

In short, the discrepancy does not happen evenly across all tokens. It mainly appears in certain types of tokens—especially those involving mathematical computation—where the difference becomes noticeably larger.

Table 13. The top 50 tokens ranked by routing discrepancy

Rank	Token ID	Token String	Mean Token-Level Diff Router
1	15	"0"	16.9834
2	37018	"frac"	16.6235
3	248	(unvisualizable)	16.5253
4	50853	"cdot"	15.9350
5	230	(unvisualizable)	15.8782
6	20	"5"	15.7010
7	116	(unvisualizable)	15.6420
8	18	"3"	15.6307
9	23	"8"	15.5839
10	19	"4"	15.3182
11	21	"6"	15.2691
12	17	"2"	15.2680
13	22	"7"	15.0927
14	3245	" least"	15.0805
15	16	"1"	14.8400
16	94	(unvisualizable)	14.6765
17	258	"in"	14.4592
18	15896	(unvisualizable)	14.3470
19	24	"9"	14.3465
20	104	(unvisualizable)	13.2339
21	79075	"boxed"	12.9715
22	944	"t"	12.6392
23	31807	" ₂ "	12.6165
24	26888	"sqrt"	11.9509
25	15940	"sin"	11.5561
26	15170	"{}"	11.4702
27	35702	"{\\""	9.8007
28	9407	"cos"	9.6640
29	14	"/"	8.7482
30	72	"i"	8.7010
31	62	"_"	8.5313
32	272	" c"	8.4916
33	90	"{"	8.4155
34	89	"z"	8.1327
35	33	"B"	8.0483
36	2086	" second"	7.9709
37	11067	" sides"	7.8640
38	77	"n"	7.8423
39	34	"C"	7.7431
40	88	"y"	7.6464
41	89669	"o"	7.5362
42	48245	" π "	7.5002
43	12	"_"	7.4991
44	65	"b"	7.4563
45	10	"+"	7.4212
46	1242	"sum"	7.4100
47	425	" B"	7.3921
48	74	"k"	7.3901
49	30986	" ₁ "	7.3890
50	198	"\n"	7.3686



Figure 10. Visualization of routing discrepancy for the response segment, where red indicates large routing discrepancy and green indicates small routing discrepancy

M. Per-Layer Routing Divergence between Training and Inference

To better understand how the routing inconsistency flows across layers, we measure the routing difference at every layer of Qwen3-30B-A3B. For each layer, we compute the mean difference (*Mean Diff*) between the routing distributions during training and inference. The results are shown in Figure 11.

As shown in Figure 11, the divergence is small in the shallow layers (0.0193 at layer 0), grows with depth, reaches its peak at layer 36 (0.2104), and then drops in the last few layers.

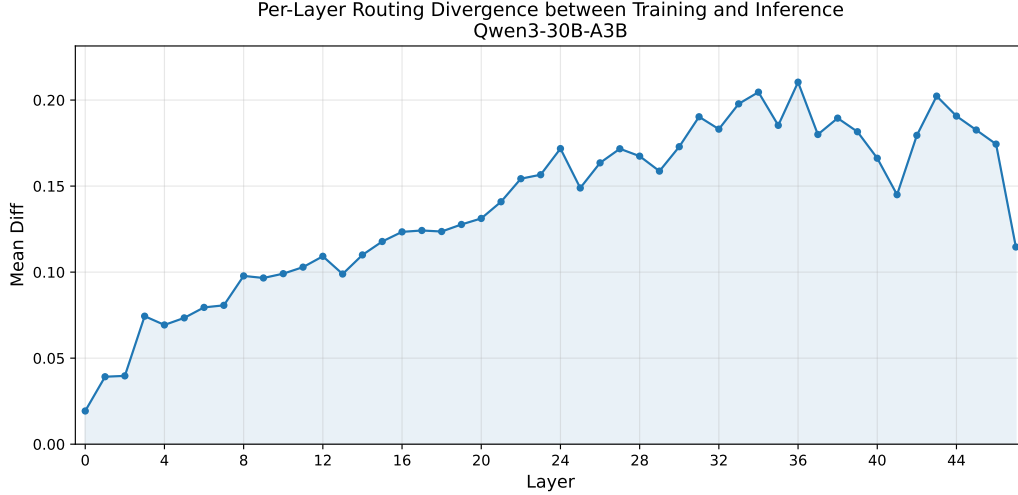


Figure 11. Per-layer routing divergence (Mean Diff) between training and inference for Qwen3-30B-A3B across all 48 layers.

N. Influence of Model Size on Training-Inference Discrepancy

We examine whether model size affects the training-inference discrepancy. We evaluate Qwen3 models of sizes 0.6B, 1.7B, 4B, 8B, 32B, 30B-A3B, and 30B-A3B + R3, including both Thinking and Base variants. The results are shown in Table 14.

The results show that: (1) no clear pattern emerges between model size and the discrepancy among dense models; (2) the MoE model (30B-A3B) shows larger discrepancy than all dense models; (3) applying R3 reduces the MoE discrepancy to levels comparable with dense models.

Table 14. Training-inference discrepancy across different model sizes. KL denotes KL divergence and $F(\tau=2)$ denotes the F metric with temperature $\tau = 2$.

Model	KL (Base)	KL (Thinking)	$F(\tau=2)$ (Base)	$F(\tau=2)$ (Thinking)
0.6B	6.631e-04	7.549e-04	4.294e-06	1.767e-05
1.7B	1.028e-03	9.389e-04	6.371e-06	6.185e-05
4B	6.037e-04	6.446e-04	1.868e-05	1.066e-05
8B	9.834e-04	6.471e-04	1.575e-05	8.651e-06
32B	—	8.709e-04	—	1.428e-05
30B-A3B	1.339e-03	1.548e-03	2.992e-04	3.377e-04
30B-A3B + R3	5.303e-04	7.347e-04	8.860e-06	1.270e-05

O. Training Hyperparameters

Table 15. Default Training hyperparameters of Reinforcement Learning.

Hyperparameter	Value
Batch Size	256
Group Size	8
Mini Step	8 or 1
Max Prompt Length	2048
Max Response Length	30720
Optimizer	AdamW
Weight Decay	0.1
Learning Rate	3×10^{-6} or 1×10^{-6}
Learning Rate Scheduler	Constant
Beta1	0.9
Beta2	0.999
Epsilon	1×10^{-8}
Max Gradient Norm	1.0
DAPO Clip-low	0.20
DAPO Clip-high	0.27
GSPO Clip-low	3e-4
GSPO Clip-high	4e-4
TIS Clip	2.0